M.PRANATI
192312298

ITA0614 – MACHINE LEARNING

CO4 AT2

INDUSTRY PROBLEM SOLVING

1. A telecommunications company wants to recommend suitable data plans to customers based on their usage patterns. Design a solution using K-Nearest Neighbour or Case-Based Learning. Explain how similarity between users is measured, how recommendations are generated, and how the system adapts to changing user behavior over time.

1. Introduction

Telecommunication companies offer different types of mobile data plans to satisfy the requirements of different customers. Customers may have different usage patterns depending on their internet browsing, video streaming, social media usage, online gaming, voice calls, and messaging habits. Selecting the most suitable data plan manually can be difficult because customer requirements continuously change.

Machine Learning can be used to automatically recommend suitable data plans based on historical customer behaviour. One suitable technique for this purpose is the K-Nearest Neighbour (KNN) algorithm. KNN is a supervised machine learning algorithm that makes predictions by finding customers who have usage patterns similar to a new customer.

The basic idea is that customers with similar usage behaviour are likely to prefer similar data plans. For example, a customer who uses a large amount of mobile data for video streaming is likely to require a high-data or unlimited-data plan. Similarly, a customer who uses only a small amount of data may be better suited to a basic or low-cost plan.

The proposed system collects customer usage information, measures the similarity between customers, identifies the nearest customers using KNN, and recommends the data plan selected by the majority of similar customers. The system can also continuously update its knowledge using recent customer behaviour.

2. User Data

The system collects features such as:

- Monthly data usage (GB)
- Number of voice calls
- Call duration
- Number of SMS
- Video streaming hours
- Social media usage
- Current monthly bill

For example:

Customer	Data (GB)	Calls	Streaming (hrs)	Plan
A	5	80	10	Basic
B	8	100	15	Standard
C	25	150	40	Premium
D	22	140	35	Premium

3. Measuring Similarity

When a new customer enters their usage details, the system calculates the distance between that customer and existing customers.

The Euclidean distance can be used:

$$d=(x1-y1)2+(x2-y2)2+\cdots+(xn-yn)2$$

A smaller distance means that two customers have more similar usage patterns.

Before calculating distance, features should be normalized because data usage, call counts, and other values have different scales.

4. Generating Recommendations

The system selects the K nearest customers to the new customer.

For example, if K = 5, the five customers having the smallest distances are selected. If most of these customers use the Premium Plan, the system recommends the Premium Plan to the new customer.

The recommendation process is:

New Customer Data → Normalize Features → Calculate Distances → Find K Nearest Users → Majority Voting → Recommend Data Plan

The system can also recommend the top 2–3 plans based on the percentage of nearby customers using each plan.

5. Case-Based Learning

The same system can use Case-Based Learning, where previous customer cases are stored. When a new customer arrives, the system searches for similar historical cases and uses their successful plans as recommendations.

For example, if previous customers with 20–25 GB usage and high streaming consumption mostly selected a Premium plan, the system can recommend that plan to a new customer with similar behaviour.

6. Adapting to Changing User Behaviour

Customer usage changes over time. Therefore, the system should continuously update its customer database.

- Store recent usage patterns regularly.
- Give greater importance to recent usage than old usage.
- Recalculate recommendations whenever usage changes significantly.
- Add new customer cases and successful recommendations to the database.
- Remove or reduce the importance of outdated customer behaviour.
- Periodically retrain or update the KNN model.

For example, a customer who previously used 5 GB/month may start using 25 GB/month because of increased video streaming. The system detects this change and can automatically recommend upgrading from a Basic plan to a Premium plan.

7. Advantages

- Simple and easy to implement.
- Provides personalized recommendations.
- Does not require complex model training.
- Can adapt to new customer data.
- Uses real customer behaviour to make recommendations.

8. Conclusion

A KNN-based recommendation system can effectively help telecommunications A telecommunications company can use K-Nearest Neighbour (KNN) to develop an intelligent and personalized data-plan recommendation system. The system collects information such as monthly data usage, call frequency, streaming hours, social media usage, and billing information. After preprocessing and normalization, the system calculates the similarity between a new customer and existing customers using Euclidean distance.

The K nearest customers are then selected, and majority voting is used to identify the most suitable data plan. Case-Based Learning can also be incorporated by storing previous customer cases and using similar successful cases for future recommendations.

Most importantly, the system can continuously adapt to changing customer behaviour. By regularly updating usage information, giving importance to recent behaviour, and using customer feedback, the recommendation can change when the customer's requirements change.

Thus, a KNN-based system can provide accurate, personalized, flexible, and cost-effective data-plan recommendations, benefiting both customers and telecommunications companies.

2. A smart city system aims to predict traffic flow and congestion levels using data from sensors, weather, and time of day. Design a solution using Locally Weighted Regression or KNN. Explain how the model captures dynamic patterns, how predictions are made in real-time, and how scalability is maintained with large volumes of data.

1. Introduction

A smart city needs an intelligent traffic management system to predict traffic flow and congestion. Traffic conditions depend on many factors such as the number of vehicles, vehicle speed, weather, time of day, day of the week, and special events.

Machine Learning can be used to predict traffic conditions using historical and real-time data. K-Nearest Neighbour (KNN) is a suitable method because it predicts the current traffic condition by finding historical situations that are most similar to the current situation.

The system collects data from traffic sensors, weather sensors, GPS devices, and cameras. Based on this information, it predicts the traffic flow and congestion level.

2. Data Used for Prediction

The system can use the following inputs:

- Average vehicle speed
- Road occupancy
- Temperature
- Rainfall
- Time of day
- Day of the week
- Previous traffic flow
- Holiday or working day

The output can be classified as:

- Low congestion
- Medium congestion
- High congestion

3. Proposed KNN Solution

The working process of the system is:

Sensor Data → Data Preprocessing → Feature Normalization → KNN Algorithm → Similarity Calculation → Find K Neighbours → Predict Traffic → Congestion Alert

First, real-time data is collected from different sensors. The data is cleaned and normalized. KNN then compares the current traffic condition with historical traffic records.

The Euclidean distance can be used to measure similarity:

$$d=(x1-y1)2+(x2-y2)2+\cdots+(xn-yn)2$$

A smaller distance means that the two traffic conditions are more similar.

4. Capturing Dynamic Traffic Patterns

Traffic patterns change depending on the time and situation. For example, traffic is usually higher during 8–10 AM and 5–8 PM because of office and school timings.

KNN captures these patterns by comparing the current situation with similar historical situations.

For example, if the current condition is:

- Monday
- 8:30 AM
- Heavy rainfall
- High vehicle count
- Low average speed

KNN searches for previous traffic records having similar conditions. If most of those records had high congestion, the system predicts high congestion.

This makes the system capable of handling different traffic patterns instead of using a fixed prediction.

5. Real-Time Prediction

The system continuously receives sensor data at regular intervals, such as every one or five minutes.

For example:

Traffic Sensors

↓

Real-Time Data

↓

Preprocessing

↓

KNN Prediction

↓

Congestion Level

↓

Traffic Alert / Control

Suppose the sensors detect an increase in vehicle count and a decrease in average speed. The KNN model compares this information with historical data.

If the nearest five traffic records contain:

- High congestion – 4 records
- Medium congestion – 1 record

6. Locally Weighted Regression

Another method that can be used is Locally Weighted Regression (LWR). It gives greater importance to data points that are more similar to the current traffic situation. For example, when predicting traffic at 8 AM on a rainy day, traffic records from previous rainy mornings receive higher importance than records from late-night periods. Thus, LWR can capture local and changing traffic patterns effectively.

7. Scalability with Large Data

Smart cities may have thousands of sensors generating large amounts of data. To maintain scalability, the system can use:

- Cloud computing for storing and processing large datasets.
- Edge computing to process some data near the sensors.
- Data aggregation to combine sensor readings into 1-minute or 5-minute intervals.
- Efficient KNN search methods such as KD-Trees to find neighbours faster.
- Distributed processing to process data using multiple computers.

These techniques reduce processing time and allow the system to handle large amounts of traffic data.

8. Advantages

- Provides real-time traffic prediction.
- Detects changing traffic patterns.
- Uses weather and time information.
- Helps reduce traffic congestion.
- Supports intelligent traffic signal control.
- Helps drivers select alternate routes.
- Can be updated with new traffic data.
- Suitable for smart-city applications.

9. Conclusion

KNN and Locally Weighted Regression can be effectively used for predicting traffic flow and congestion in smart cities. KNN identifies historical traffic situations similar to the current situation and uses them to make predictions. LWR gives greater importance to nearby and relevant traffic patterns. Real-time sensor data allows the system to continuously update predictions. Cloud computing, edge computing, data aggregation, and efficient search techniques help the system handle large volumes of traffic data.